

Auditoría de acceso, licenciamiento y flujo de datos

Plataforma EcoNexo · rama **main** · entorno de producción **econexo.onrender.com** · 21/08/2026

Alcance y método

Este informe documenta las inconsistencias detectadas al revisar tres cuestiones: quién puede entrar a la plataforma, cómo se aplican los límites de la licencia, y si los datos que la plataforma procesa y muestra provienen de fuentes reales.

El relevamiento se hizo leyendo el código de la rama **main**, contrastándolo con el blueprint de despliegue **render.yaml** y verificando contra el entorno productivo los endpoints accesibles sin credenciales. Los hallazgos marcados como **Resuelta** fueron corregidos en el mismo cambio que acompaña este informe; los demás quedan abiertos y requieren una decisión de producto o de infraestructura.

Una limitación explícita: no se verificaron los valores de las variables de entorno marcadas **sync: false** en Render (credenciales de NASA FIRMS y Copernicus), porque no son legibles desde el código ni desde la API sin acceso al panel. Los hallazgos que dependen de ellas se señalan como pendientes de confirmación.

Resumen

#	Hallazgo	Severidad	Estado
A1	Cualquier persona podía registrarse y entrar de inmediato	Alta	Resuelta
A2	El alta con Google era una segunda puerta sin control	Alta	Resuelta
A3	Toda acción de administración general sobre otra organización fallaba	Alta	Resuelta
A4	El arranque del administrador general es código muerto	Media	Abierta
B1	Ningún sensor físico puede entregar datos a la plataforma desplegada	Alta	Abierta
B2	El planificador del pipeline nunca se ejecuta	Alta	Abierta
B3	El feed en tiempo real no emite ningún mensaje	Alta	Abierta
B4	La confianza de las alertas usa un valor fijo, no un modelo	Alta	Parcial
B5	Las fotos de reportes ciudadanos se descartan en silencio	Media	Abierta
B6	Focos de calor y capas satelitales fallan sin aviso	Media	Abierta
B7	Los nodos virtuales conviven con los reales sin distinción	Media	Abierta
C1	Tope de 1 a 4 informes mensuales según plan	Media	Resuelta
C2	La barra de uso marcaba 100% en recursos sin tope	Baja	Resuelta
D1	El panel público de Estado reportaba una falla inexistente	Media	Resuelta
D2	La portada describe capacidades que el despliegue no tiene	Alta	Abierta

La severidad refleja la distancia entre lo que la plataforma afirma hacer y lo que efectivamente hace en producción, no la dificultad de corregirlo.

A · Control de acceso

A1

Cualquier persona podía registrarse y entrar de inmediato

Resuelta

POST /auth/register creaba la organización con **is_active = true**, creaba al usuario como **admin** y devolvía un token de sesión en la misma respuesta. No existía ninguna verificación de correo, aprobación manual ni pago previo. Con un correo válido y una contraseña de ocho caracteres se obtenía una organización propia y acceso al centro de comando, incluida una suscripción *sandbox* de catorce días que se creaba sola con **activation_source = automatic_registration**.

```
INSERT INTO organizations (... is_active) VALUES (... , true)
INSERT INTO users (... role ...) VALUES (... , 'admin', ...)
return _session(row, is_new_user=True)
```

Impacto. La plataforma no tenía puerta comercial: el licenciamiento existía en el modelo de datos pero no condicionaba el acceso.

Estado. El alta institucional ahora crea la organización en **pending** con **is_active = false** y responde **202** con un acuse, sin token. El acceso lo habilita administración general desde la consola. El teléfono de contacto pasó a ser obligatorio y se expone como enlace de WhatsApp. Las cuentas comunitarias de EcoNexoFol siguen siendo gratuitas e inmediatas, por ser el nivel gratuito declarado del producto.

A2

El alta con Google era una segunda puerta sin control

Resuelta

POST /auth/google con **mode = register** repetía exactamente la misma lógica: organización activa, usuario administrador y sesión inmediata. Cerrar solamente el alta por email habría dejado el hueco abierto. En el entorno actual la vía está inerte porque **GOOGLE_CLIENT_ID** está vacío, pero se habría reabierto el día que se configure.

Impacto. Un control de acceso que depende de que una integración siga sin configurarse no es un control de acceso.

Estado. El alta institucional por Google fue eliminada y deriva al formulario por email, que es el que pide teléfono. El ingreso por Google de cuentas ya existentes sigue funcionando y ahora respeta el estado **pending**.

A3

Toda acción de administración general sobre otra organización fallaba

Resuelta

La tabla **audit_events** tenía una clave foránea compuesta (**org_id, user_id**) contra **users(org_id, id)**, que exige que el actor pertenezca a la organización auditada. Administración general es cruzada por definición: aprueba, suspende y renombra organizaciones que no son la suya. Cada una de esas acciones violaba el constraint. Se detectó al probar el circuito de aprobación de altas de punta a punta.

```
asyncpg.exceptions.ForeignKeyViolationError: insert or update on table
"audit_events" violates foreign key constraint
"fk_audit_events_org_user"
```

Impacto. Lo peor era la forma de fallar: el **UPDATE** se aplicaba y recién después fallaba la auditoría, sin transacción que los uniera. La organización quedaba efectivamente aprobada, pero el administrador veía un error 500 y no tenía forma de saber si la acción había surtido efecto.

Estado. La migración **19** reemplaza el FK compuesto por uno simple sobre el actor. Se conserva la integridad referencial del usuario y se abandona la exigencia de que actor y organización coincidan, que la consola de plataforma no puede cumplir. La migración limpia primero los actores huérfanos, porque el constraint anterior era **NOT VALID** y las filas viejas nunca se verificaron.

ensure_platform_admin está definida en **platform_admin.py** y no se invoca desde ningún lado. Es el mismo patrón que el planificador del pipeline (B2): una función completa que nadie ejecuta. En consecuencia, cuatro variables declaradas en **render.yaml** no tienen ningún efecto: **PLATFORM_ADMIN_BOOTSTRAP_ENABLED**, **PLATFORM_ADMIN_INITIAL_PASSWORD**, **PLATFORM_ADMIN_RESET_INITIAL_PASSWORD** y **PLATFORM_ADMIN_FORCE_PASSWORD_CHANGE**.

Impacto. Hoy no rompe nada, porque los privilegios de administración general se resuelven por la lista **PLATFORM_ADMIN_EMAILS** y no por una marca en la base. Pero deja cuatro perillas en el panel de Render que un operador puede creer que hacen algo. Conviene conectarla o quitar las variables.

B · Flujo de datos procesados

Esta sección responde a la pregunta de si los datos que la plataforma muestra son reales. La respuesta corta: las fuentes que sí funcionan son reales, pero varias de las que la interfaz presenta como activas no están conectadas en producción, y ninguna de esas ausencias se comunica al usuario.

B1

Ningún sensor físico puede entregar datos a la plataforma desplegada

Alta

Se rastrearon todos los caminos de escritura sobre la tabla **readings**. En el código desplegado existe uno solo: **refresh_open_meteo_device**, que consulta la API de Open-Meteo. Los otros dos escritores del repositorio no corren en producción: **services/ingest/index.js** no figura en render.yaml, y **app/demo.py** es un script manual. No existe endpoint HTTP para que un dispositivo publique lecturas: el router de dispositivos solo expone **GET /devices/{id}/readings**. El puente MQTT está apagado por **MQTT_ENABLED=false** y, además, solo retransmite a WebSocket: nunca escribe en la base.

```
render.yaml despliega 2 servicios: econexo (API) y econexo-web (estatico).
docker-compose.yml define 8: postgres, mosquitto, minio, api, ingest,
notify, anomaly, satellite, web, simulator.
```

Impacto. La totalidad de las lecturas que hoy se muestran como telemetría proviene de un modelo numérico de pronóstico meteorológico, no de mediciones de campo. Open-Meteo es una fuente real y legítima, pero es un modelo, no un sensor. La distinción es sustantiva para un informe institucional que se presenta como evidencia.

B2

El planificador del pipeline nunca se ejecuta

Alta

La función **pipeline_scheduler** está definida en **telemetry_pipeline.py** pero no se la invoca desde ningún lado: el **lifespan** de la aplicación solo lanza la tarea del puente MQTT. Es código muerto. En consecuencia, los campos **auto_run** e **interval_minutes** se guardan en la base y se editan desde la interfaz, pero no producen ningún efecto.

```
bridge = asyncio.create_task(mqtt_bridge(_stop)) # unica tarea de fondo
grep pipeline_scheduler --> 1 sola aparicion: su propia definicion
```

Impacto. El panel de telemetría ofrece un control rotulado *Ejecución automática · Scheduler liviano dentro de la API* que el operador puede activar y configurar, y que no hace nada. El pipeline únicamente corre cuando alguien pulsa *Ejecutar* manualmente.

B3

El feed en tiempo real no emite ningún mensaje

Alta

manager.broadcast se invoca desde un único lugar, **_dispatch**, que a su vez solo se alcanza desde el puente MQTT. Con **MQTT_ENABLED=false** el puente sale de inmediato por su primera guarda. La función **publish** también retorna sin hacer nada bajo esa misma bandera. El endpoint **/ws** acepta la conexión y valida el token correctamente, y después queda en silencio indefinidamente.

Impacto. La promesa de *decisión en tiempo real* no tiene sustento operativo en el despliegue actual. El cliente se conecta, no recibe nada, y no hay forma de distinguir eso de *no hay novedades*.

B4**La confianza de las alertas usa un valor fijo, no un modelo****Parcial**

anomaly_score consulta el servicio de anomalías por HTTP y, ante cualquier fallo, degrada a **0.5**. Ese servicio no está desplegado en Render y **ANOMALY_SERVICE_URL** no está definida, con lo cual se usa el valor por omisión **http://localhost:8100**: la llamada siempre falla. El **0.5** resultante se convierte en **sensor_score**, entra a **correlate()** con un peso de 0.42 y termina siendo la **confianza** que se muestra en cada alerta.

```
sensor_score = max(matched_scores, default=0.65)
SOURCE_WEIGHTS = {'sensor': 0.42, 'satelite': 0.32, 'ciudadano': 0.18}
```

Impacto. La confianza que acompaña a cada alerta se lee como la salida de un modelo y es, en los hechos, una constante. Además, la función ignoraba **ANOMALY_ENABLED=false** e intentaba la llamada igual, pagando hasta 3 segundos de espera por cada condición evaluada; con **PIPELINE_MAX_DEVICES_PER_RUN=100** eso podía sumar minutos muertos por corrida.

Estado. Se corrigió el desperdicio de tiempo: **anomaly_score** ahora respeta la bandera y devuelve el score neutro sin salir a la red. **Lo de fondo sigue abierto:** mientras el servicio no esté desplegado, la confianza sigue siendo un valor fijo y conviene no presentarla como si fuera inferida.

B5**Las fotos de reportes ciudadanos se descartan en silencio****Media**

Con **S3_ENABLED=false**, que es la configuración productiva actual, **put_photo** retorna **None** antes de intentar nada. El router de reportes guarda el registro con **photo_url** en nulo y responde **201**. El ciudadano adjunta una foto, la aplicación confirma el envío, y la foto no queda en ninguna parte.

Impacto. Se pierde evidencia que el emisor cree haber aportado. En un flujo de reportes ciudadanos con puntaje de reputación, la foto suele ser el elemento que sostiene la validación posterior.

B6**Focos de calor y capas satelitales fallan sin aviso****Media**

refresh_firms retorna **(0, 0)** si **NASA_FIRMS_KEY** está vacía, sin registrar advertencia ni marcar la corrida como degradada. La clave está declarada como **sync: false** en **render.yaml**, igual que las credenciales de Copernicus, de modo que su presencia depende de una carga manual en el panel que no pudo verificarse desde fuera. En paralelo, **COPERNICUS_ENABLED_BY_DEFAULT** está en **true**, con lo cual la capa se anuncia como activa aunque no haya credenciales que la sostengan.

Impacto. Si las credenciales no están cargadas, la ausencia de focos de calor es indistinguible de *no hubo focos*. Conviene confirmarlo en el panel de Render y, en cualquier caso, hacer que el estado de configuración de cada fuente sea visible en la interfaz.

B7**Los nodos virtuales conviven con los reales sin distinción****Media**

POST /pipeline/bootstrap crea hasta seis dispositivos llamados *Nodo virtual N*, con etiquetas **virtual** y **open-meteo** y **telemetry_mode = open_meteo**. Son puntos de consulta a un modelo meteorológico, no equipos instalados. El etiquetado es correcto, pero en el mapa aparecen como cualquier otro dispositivo y consumen cupo de **max_devices**; solo el globo emergente menciona el **telemetry_mode**.

Impacto. Un usuario que no conozca el detalle interno lee *6 nodos en línea* como seis equipos desplegados en territorio.

C · Licenciamiento

C1	Tope de 1 a 4 informes mensuales según plan	Resuelta
----	---	----------

enforce_monthly_report_limit contaba los informes del mes en curso y respondía **402** al alcanzar **max_reports_per_month**. Los topes iban de 1 informe en los planes de evaluación y diagnóstico a 4 en Piloto y SaaS Municipal, 12 en Provincia y 2 en Academia.

Estado. Se eliminó el tope de los siete planes y se quitó la verificación de cantidad. La licencia activa sigue siendo obligatoria para emitir informes: cambió el racionamiento por cantidad, no la puerta comercial. La migración limpia además el *entitlement* de las filas ya guardadas, para que la consola no muestre un límite que nadie aplica.

C2	La barra de uso marcaba 100% en recursos sin tope	Resuelta
----	---	----------

El panel de suscripción calculaba el porcentaje como **current ? 100 : 0** cuando no había límite definido. Un recurso ilimitado con cualquier consumo se dibujaba con la barra llena, que es exactamente la señal visual de *cupó agotado*.

Estado. Sin tope la barra queda vacía y la cifra se acompaña de *sin límite*.

D · Coherencia entre lo que se afirma y lo que corre

D1	El panel público de Estado reportaba una falla inexistente	Resuelta
----	--	----------

SystemStatus consulta */health*, */ready* y */territory/boundary-status*. El endpoint */ready* no existía en la API: se verificó contra producción y devolvía **404**. Como el componente marca ese control como **error** cuando la respuesta no es correcta, la página pública de Estado mostraba de forma permanente *Base y PostGIS · Readiness incompleto* y un encabezado global de *Revisión necesaria*, con todo funcionando.

```
GET https://econexo.onrender.com/ready --> 404 {"detail":"Not Found"}
```

Estado. Se agregó */ready*, que comprueba conectividad con la base, presencia de PostGIS y de la guarda territorial, y responde **503** si algo falta. */health* ahora incluye el territorio que la interfaz ya intentaba leer.

D2	La portada describe capacidades que el despliegue no tiene	Alta
----	--	------

Los tres reclamos de la portada se contrastaron con el despliegue. *Integra sensores IoT*: no hay vía de ingesta para un sensor físico (B1). *< 5 min · objetivo de detección*: nada corre de forma automática, porque el planificador no se ejecuta (B2), de modo que la latencia real depende de que alguien pulse un botón. *PostGIS + IA · correlación espacial*: la parte PostGIS es cierta y funciona; la parte de IA es el servicio de anomalías, que no está desplegado y devuelve una constante (B4).

Impacto. Es el hallazgo con mayor exposición: son afirmaciones dirigidas a compradores institucionales, en una plataforma cuyo producto es la trazabilidad de la evidencia. Se recomienda ajustar el texto a lo que el despliegue sostiene hoy, o desplegar las piezas faltantes antes de sostener el texto.

Recomendaciones, por orden de prioridad

#	Acción	Hallazgos
1	Decidir explícitamente qué se afirma en la portada y en el material comercial. Es una decisión de producto, no técnica, y condiciona todo lo demás.	D2
2	Arrancar pipeline_scheduler en el <i>lifespan</i> , o quitar de la interfaz el control de ejecución automática. Hoy el usuario configura algo inexistente.	B2
3	Confirmar en el panel de Render si NASA_FIRMS_KEY y las credenciales de Copernicus están cargadas, y hacer visible en la interfaz el estado de configuración de cada fuente.	B6
4	Definir el camino de ingesta real: desplegar el servicio ingest con un broker MQTT, o exponer un endpoint autenticado de publicación de lecturas.	B1
5	Distinguir en el mapa y en los conteos los nodos modelados de los equipos físicos, y excluirlos del cupo de dispositivos de la licencia.	B7
6	Avisar al ciudadano cuando la foto no puede almacenarse, o habilitar almacenamiento de objetos.	B5
7	Desplegar el servicio de anomalías, o dejar de presentar la confianza como un valor inferido mientras devuelva una constante.	B4
8	Emitir por WebSocket los eventos que hoy solo viajan por MQTT, para que el feed en vivo funcione sin depender del broker.	B3
9	Conectar ensure_platform_admin al arranque, o quitar de render.yaml las cuatro variables que hoy no hacen nada.	A4

Los hallazgos marcados como resueltos se corrigieron en el cambio que acompaña a este informe e incluyen la migración **18_access_requests_and_phone.sql**. Los abiertos requieren una decisión previa sobre qué debe sostener la plataforma antes de programar la corrección.